



СИСТЕМНИЙ АУДИТ

ProjectCentrum MES

Повний аналіз системи від А до Я

ЗАГАЛЬНА ОЦІНКА СИСТЕМИ

73%

Система функціональна, але має критичні вузькі місця

ДАТА АУДИТУ

29.08.2026

ТАБЛИЦЬ БД

40+

МІГРАЦІЙ

66

МОДУЛІВ

14

ТРИГЕРІВ БД

15+

RPC ФУНКЦІЙ

25+



Зміст

Структура звіту

1	Виконавче резюме та загальна оцінка	стор. 3
2	Архітектура системи (огляд)	стор. 4
3	Потік виробництва — деталізований аналіз	стор. 5
4	Модуль Склад (Warehouse V2)	стор. 6
5	Буферна Зона (БЗ) — облік та резервування	стор. 7
6	Модуль Брак та ВКЯ	стор. 8
7	Модуль Повернення деталей у наряди	стор. 9
8	Цехові термінали (Shop1 / Shop2)	стор. 10
9	Аналітика та звітність	стор. 11
10	Вузькі місця та технічний борг	стор. 12
11	Рекомендації (пріоритети)	стор. 13
12	Зведена таблиця оцінок по модулях	стор. 14



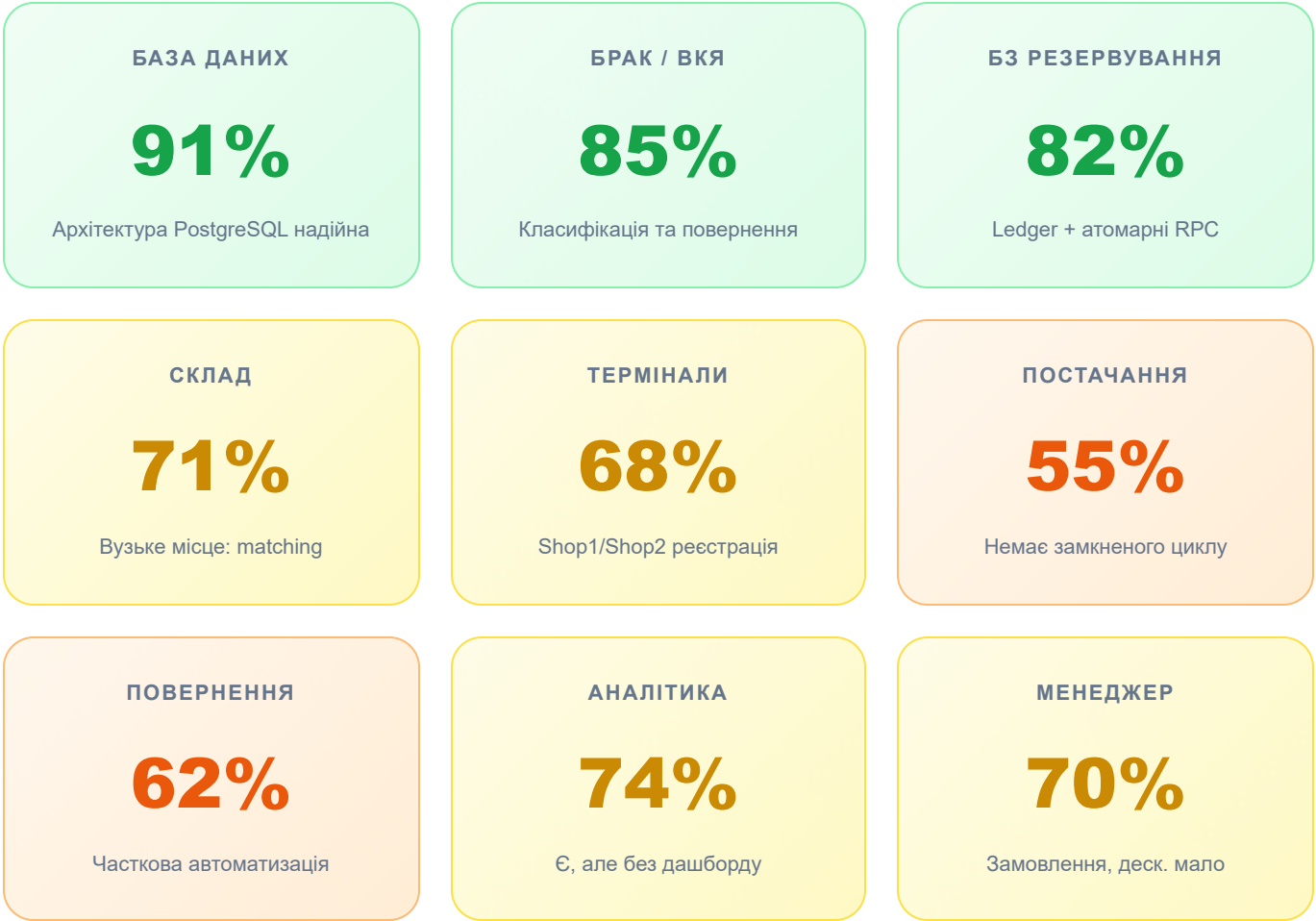
Як читати цей звіт

Кожен модуль оцінено за шкалою **0–100%**. Відсоток відображає якість реєстрації та коректність руху даних (деталей, матеріалів, браку, повернень). Критичні вузькі місця позначено червоним кольором і містять конкретні технічні причини та рекомендації.



Розділ 1

Виконавче резюме та загальна оцінка



🔑 Ключові висновки

✓ Що працює добре (сильні сторони)

- **Архітектура бази даних** — 40+ таблиць, 66 міграцій, PostgreSQL тригери для автоматичної синхронізації. Дані не губляться.
- **BZ Ledger** — повний аудиторський журнал руху деталей у Буферній Зоні. Кожна зміна фіксується з балансами ДО і ПІСЛЯ.
- **Класифікація браку (ВКЯ)** — 4 категорії, повернення до маршруту, відновлення — атомарні RPC функції, без ризику дублювання.
- **Фулфілмент черга (пакування/відвантаження)** — є bounded query, atomic packing slip counter. Немає дублікатів квитків.
- **Real-time синхронізація** — Supabase WebSockets, зміни миттєво видно на всіх вкладках.
- **Snapshot системи** — наряди зберігають стан замовлення в момент створення. Зміни специфікацій не ламають активні наряди.

🔥 Критичні вузькі місця (потребують негайної уваги)

- **Матч запитів до залишків (Warehouse)** — matching по текстових назвах є крихким. «Лист T300 (4мм)» може не знайти «Лист T300-4mm». Втрачаються видачі.
- **reserved_qty в inventory** — тригер reconcile_inventory_reserve пахує лише status='issued' запити. Заявки у статусі 'pending' не резервують матеріал.
- **Постачання без замкненого циклу** — модуль SupplyModule не має автоматичного зв'язку з фактичним надходженням на склад. Прийомка ручна.
- **Велика кількість scratch-файлів (789 штук)** — ознака того, що система часто «виправлялась вручну». Системний технічний борг.
- **Подвійні версії модулів** — існують MasterModule.jsx і MasterModule_v3.jsx, EngineerModule.jsx і EngineerV2Module.jsx. Незрозуміло який є актуальним.



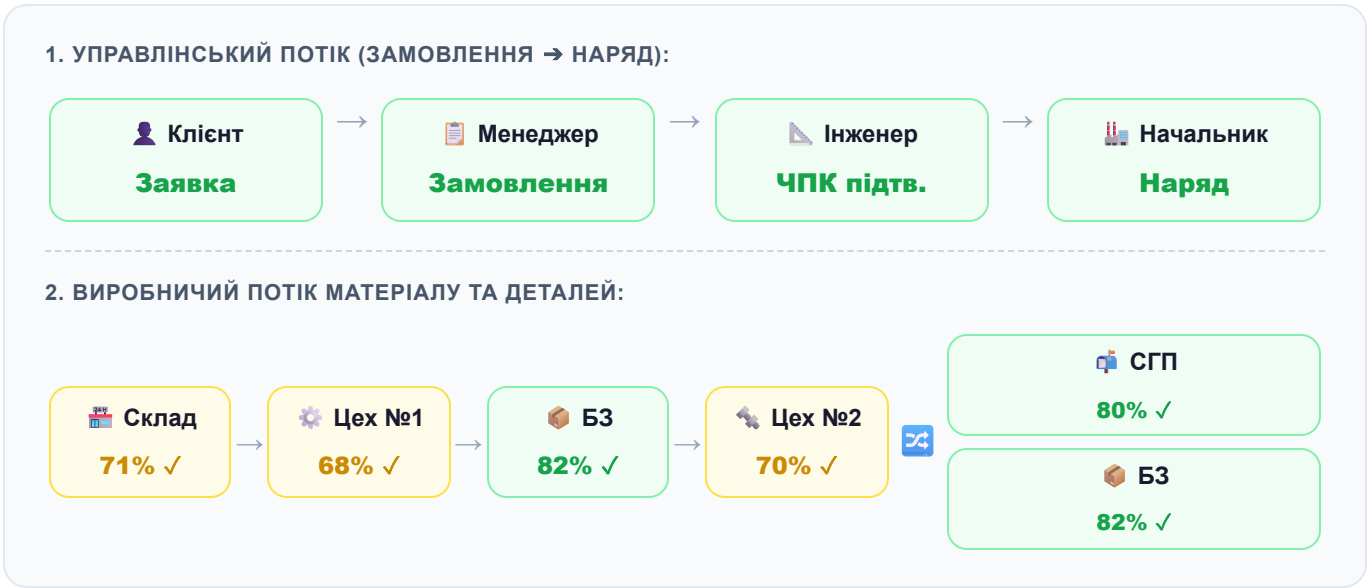
Розділ 2

Архітектура системи

Технологічний стек

ШАР	ТЕХНОЛОГІЯ	СТАТУС	ПРИМІТКА
Frontend	React.js + Vite	✓ СТАБІЛЬНО	Компонентна архітектура. Є дублювання версій.
Backend / DB	Supabase (PostgreSQL 15)	✓ НАДІЙНО	RLS увімкнено. 66 міграцій без конфліктів.
Real-time	Supabase WebSockets	✓ АКТИВНО	Змінна видна одразу на всіх вкладках.
Аутентифікація	system_users (власна)	⚠ КАСТОМНА	Не через Supabase Auth. Ризики безпеки.
Деплой	Vercel	✓ ОК	vercel.json присутній.
PDF / Друк	Browser Print CSS	⚠ ОБМЕЖЕНО	Немає сервер-сайдного PDF генератора.

Схема потоку даних



Ключові таблиці бази даних (36 основних)

ТАБЛИЦЯ	ПРИЗНАЧЕННЯ	ЗВ'ЯЗКИ
orders	Замовлення (з order_num, customer, deadline)	→ tasks
tasks	Наряди. Містять plan_snapshot (JSONB)	→ work_cards, material_requests
work_cards	Робочі карти з операцією, статусом, кількістю	→ work_card_history
work_card_history	Аудитний лог кожного переходу карти (stage_name, qty_completed, scrap_qty)	Source of truth
work_card_flow_totals	Pre-aggregated підсумки по kartax (good/bz/scrap)	Синхр. тригером
inventory	Залишки всіх типів: raw, bz, wip_bz, scrap, semi	← material_requests
bz_inventory_ledger	Повний журнал руху БЗ (з балансами до/після)	← bz_inventory_reservations
scrap_classifications	Класифікація браку ВКЯ за категоріями (1-4)	← work_card_history
vkya_quality_resolutions	Рішення ВКЯ (повернення до маршруту / відновлення)	→ work_cards
nomenclatures	Каталог ТМЦ (деталі, матеріали, фрези)	Всюди
material_requests	Запити на матеріали від нарядів до складу	→ inventory
mes_counters	Атомарний лічильник (packing_slip_number)	Одиночний singleton

Розділ 3



Потік виробництва — деталізований аналіз

Етап 1 — Внесення замовлення (Менеджер)

Реєстрація замовлення

70%

Що є

Менеджер вносить замовлення з `order_num`, `customer`, `deadline`. 3 серпня 2026 додано `invoice_num` (міграція 20260818). 3 серпня 2026 вже є і CRM модуль (`leads/stages`).

Що бракує

- Немає прив'язки до специфікації виробу безпосередньо в замовленні (лише через `task→BOM`).
- CRM (`crm_leads_and_stages`) додано лише 28 серпня 2026 — дуже новий, не протестований в реальній роботі.
- Немає автоматичного сповіщення менеджера при готовності замовлення до відвантаження.

Етап 2 — Формування Наряду (Начальник цеху)

Розрахунок та генерація наряду

78%

Сильні сторони

- **Snapshot система** — при створенні наряду фіксується стан BOM, БЗ та специфікацій. Наступні зміни в каталозі не впливають на активну роботу.
- **BZ auto-deduction** — система автоматично вираховує деталі з БЗ (`bz_inventory_reservations` + `RPC reserve_bz_for_naryad`).
- **Batch logic** — підтримка `batch_index` для розбивки великих замовлень на партії.

Вузьке місце

- Якщо наряд видаляється або скасовується — BZ резервація не завжди звільняється автоматично. Функція `release_bz_reservation` є, але викликається вручну.
- Відсутній UI-індикатор у форемена для «застряглих» резервацій (`allocated` але без `task_id`).

Етап 3 — Генерація Робочих Карт

Що добре

Карти прив'язані до task_id, order_id, nomenclature_id. Є card_info (JSONB-тег рядки) для трасування походження ([VKYA_RETURN], [RESTORATION], тощо). Числова послідовність карт підтримується card_sequence.

Ризик

card_info — текстове поле з тегами у форматі [TAG_NAME:value]. Парсинг через regex є крихким. Якщо хтось введе неправильний символ у назву деталі або операції — regex зламається тихо.

Загальна оцінка маршруту деталі

 РЕЄСТРАЦІЯ ЗАПУСКУ Майстер сканує — ОК	 ФІКСАЦІЯ БРАКУ work_card_history	 ПОВЕРН. В НАРЯД Частково ручне	 БЗ ЗАРАХУВАННЯ Автоматично
---	---	--	---

Розділ 4

Модуль Склад (Warehouse V2)



Типи залишків (inventory.type)

ТИП	ПРИЗНАЧЕННЯ	СТАТУС РЕЄСТРАЦІЇ
raw / material	Сировина, листи металу	✓ КОНТРОЛЮЄТЬСЯ
bz	Буферна Зона (готові після розкрою)	✓ LEDGER + ТРИГЕР
wip_bz	БЗ зарезервована під наряд	✓ АТОМАРНО
bz_shop2	БЗ Цеху №2 (пресування/фарбування)	⚠ ЧАСТКОВО
scrap_ready	Брак очікує класифікації ВКЯ	✓ АТОМАРНО
semi	Напівфабрикати	⚠ БЕЗ LEDGER
semi_shop2	Напівфабрикати Цеху №2	⚠ БЕЗ LEDGER
sgp	Склад готової продукції	⚠ БЕЗ АВТОМАТИЗМ.

● **Критична вада: текстовий matching**

Файл `materialInventoryMatching.js` використовує `regex`-based парсинг для зіставлення запитів до залишків. Перевіряє: товщина (мм), марка (Т300/Т700), стан (підготовлений/непідготовлений).

Ризик: якщо назва введена інакше («лист 4мм» vs «Лист (4мм)»), матч не спрацює. Матеріал буде показано як «відсутній», хоча фізично він є. Це призводить до зайвих замовлень у постачання.

Рішення: перейти на `nomenclature_id` як первинний ключ матчингу (вже підтримується як запасний варіант у `inventoryMatchesRequest`).

● Резервування матеріалу (`reserved_qty`)

Тригер `reconcile_inventory_reserve` (міграція 20260818) оновлює `reserved_qty` коли змінюється `material_requests`. Але він рахує лише `status='issued'`. Замовлення зі статусом `'pending'` (ще не видані) не блокують залишок. Два форемени можуть одночасно «видати» один і той самий лист.

✓ Що добре

- Підтримка `warehouse='operational'` та інших складів. Пріоритизація `operational` в `matching`.
- `rocket_owner` — дозволяє «прив'язувати» залишки до конкретного підрозділу.
- `created_at` тепер є в `inventory` (міграція 20260726) — можливий FIFO облік.



Розділ 5

Буферна Зона (БЗ) — облік та резервування

Точність обліку БЗ

82%

Як працює БЗ-підсистема

Алгоритм резервування (атомарний RPC)

- **reserve_bz_for_naryad()** — блокує рядок через FOR UPDATE, переміщує qty з bz → wip_bz, пише в ledger. Ідемпотентно (повторний виклик повертає існуючу резервацію).
- **attach_bz_reservation_to_task()** — прив'язує operation_id до task_id після генерації карт.
- **release_bz_reservation()** — повертає qty з wip_bz → bz, позначає reserv. як released.

✓ Ledger (bz_inventory_ledger)

Кожен рух фіксується з полями: from_balance_before, from_balance_after, to_balance_before, to_balance_after. Є audit trigger audit_legacy_bz_inventory_change для прямих SQL-змін (legacy). Відкриваючий баланс (opening_balance) зафіксований при введенні ledger.

● Вузькі місця БЗ

- **Осиротілі wip_bz резервації** — якщо наряд скасовано без виклику release_bz_reservation(), залишок заморожений у wip_bz назавжди. Немає періодичного reconciliation job.
- **pocket_owner фільтр** — в reserve_bz_for_naryad() беруться лише записи де pocket_owner IS NULL OR pocket_owner = 'Не вказано'. Деталі в «кишенях» (зарезервовані іншим підрозділом) ігноруються, навіть якщо вони доступні.
- **bz_shop2 тип** — БЗ Цеху №2 не повністю покрита ledger (тригер audit_legacy_bz_inventory_change покриває bz, wip_bz, bz_shop2 — але запис іде як 'legacy_adjustment', без контексту).

Рух деталі через БЗ (повний маршрут)





Модуль Брак та ВКЯ (Відділ Контролю Якості)

Повнота обліку браку

85%

Класифікація браку (4 категорії)

КАТЕГОРІЯ	ОПИС	ЩО ВІДБУВАЄТЬСЯ
Кат. 1	Можливе виправлення без зусиль	Повертається до маршруту (return_vkya_quantity_to_route)
Кат. 2	Вимагає відновлення	Створюється vkya_restoration_card → Цех №2
Кат. 3	Складне відновлення	Відновлення або повторний цикл
Кат. 4	Незворотній брак	Списується (scrap_classification_categories). ЛИШЕ ця кат. → реальні збитки

- ✓ Що дуже добре в системі браку
- **Атомарний запис** — record_scrap_classification() перевіряє що сума категорій = сумі причин = загальній кількості. Дублювання неможливе.
 - **Тригер validate_vkya_classification_capacity** — не дозволяє класифікувати більше браку, ніж є в history запису. Безпека рахунку гарантована на рівні БД.
 - **View vkya_final_scrap_totals** — агрегує лише кат. 4, уникаючи підрахунку «відновлюваного» браку як реального збитку.
 - **Чепра vkya_classification_queue_projection** — real-time оновлення через sequence. Оператор ВКЯ бачить нові позиції миттєво.

- Що потребує покращення
- **scrap_ready тип** — деталі між Work Card History і класифікацією ВКЯ зберігаються у scrap_ready. Немає TTL або alerting якщо вони там «висять» більше N днів.
 - **Причини браку (scrap_reasons)** — каталог є, але немає UI для аналізу топ-5 причин у Директора/Майстра в режимі реального часу.
 - **Часткова класифікація** — можна класифікувати не весь брак (partial). Решта залишається pending у черзі. Без нагадувань.



Розділ 7

Повернення деталей у Наряди

Автоматизація повернень

62%

Сценарії повернення деталей



Сценарій 1: ВКЯ → Повернення до маршруту (return_vkya_quantity_to_route)

Автоматично

RPC функція визначає цільовий статус і операцію на основі stage_name запису history. Якщо карта вже у правильному статусі — merge, інакше — нова робоча карта. Інвентаризація scrap_ready списується автоматично. Запис у vkya_quality_resolutions.



Сценарій 2: ВКЯ → Відновлення → Цех №2 → БЗ

Автоматично

create_vkya_restoration_from_hold → dispatch_vkya_restoration_to_shop2 → complete_vkya_shop2_card_to_bz. Якщо джерело має task_id → деталь повертається до оригінального наряду (v_returns_to_route=true). Якщо legacy — іде в загальну БЗ.



Сценарій 3: Redo Card (переробка без ВКЯ)

Частково

Майстер може створити Redo Card ([REDO] тег у card_info) вручну. Час виконання рахується окремо від основної статистики. Але: зв'язок Redo Card → оригінальний наряд відслідковується тільки через card_info рядок, а не через FK. Звітність по Redo неповна.



Сценарій 4: Ручне коригування залишків (Manual Issue)

Ризик

Існує manual_inventory_issues таблиця (міграція 20260724). Ручні зміни не прив'язані до конкретного наряду та не відображаються у аудитному ланцюжку. Це «сліпа зона» — незрозуміло чому зменшився залишок.

● Ключова проблема повернень

У RPC return_vkya_quantity_to_route визначення цільового статусу залежить від stage_name через довгий if-elsif ланцюг. Якщо stage_name написано з помилкою, опечаткою або інакше — деталь потрапляє в 'new' статус замість правильного. Жодного fallback alerting немає.



Розділ 8

Цехові Термінали (Shop1 / Shop2)

SHOP1 TERMINAL**68%**

Розкрій. Брак ОК.

SHOP2 TERMINAL**70%**

Пресування/Фарбування

TUMBLING TERMINAL**72%**

Галтовка

Shop1 Terminal — Цех №1 (Розкрій)

Що реєструється правильно

- Запуск: майстер вказує верстат, оператора → status: 'in-progress'
- Завершення: кількість браку → work_card_history (scrap_qty, qty_completed, stage_name='Розкрій')
- Автоматичне зарахування у БЗ через trigger на work_card_history → sync_work_card_flow_totals
- Фіксація фрез (cutters_used) у history — для аналітики

Ризики Shop1

- Є окремий Shop1Terminal.jsx (278KB) і Shop1ForemanModule.jsx (139KB) — дуже великі компоненти. Складно підтримувати.
- Термінал не попереджає якщо для деталі немає ЧПК програми (перевірка у EngineerModule, але термінал автономний).
- Немає таймера на картку — невідомо скільки реально часу зайняла операція на верстаті.

Shop2 Terminal — Цех №2 (Пресування / Фарбування)

Добре

Shop2 розрізняє 'Пресування' і 'Фарбування' операції. При завершенні: деталі → bz_shop2 (або bz, або повернення до маршруту для [VKYA_SOURCE_ROUTE] карт).

Критичне вузьке місце Shop2

- **bz_shop2 без deduction-trigger** — коли деталь виходить з Цеху №2 в СГП, залишок bz_shop2 зменшується? Де це відбувається? Це незрозуміло з міграцій.
- **Перехід semi → semi_shop2** — існує, але без ledger-запису. Прозорості нема.

Галтовка / Tumbling Terminal**Галтовка — специфіка**

Є 4 под-операції: Вібростіл → Мийка → Галтовка → Сушка. Відстежується через stage_name. Повернення ВКЯ знає цей маршрут (if-elsif в return_vkya_quantity_to_route). Але відсутній ageing report по деталях у проміжних стадіях галтовки.



Розділ 9

Аналітика та Звітність



Що є в системі аналітики

ЗВІТ / ФУНКЦІЯ	RPC / VIEW	СТАТУС
Всі-часовий підсумок виробництва	mes_production_summary_rollup_all_time()	✓ РЕАЛЬНИЙ ЧАС
Щомісячний звіт MES	mes_monthly_report(p_month)	✓ ДЕТАЛЬНИЙ
Брак по категоріях	scrap_report_by_category (view)	✓ Є
Брак по причинах	scrap_report_by_reason (view)	✓ Є
Фулфілмент черга	mes_fulfillment_queue(p_queue)	✓ BOUNDED
Підсумки карт по стадіях	work_card_flow_totals (table)	✓ ТРИГЕР
Drilldown наряду	monthly_naryad_drilldown	✓ Є
Ефективність оператора	—	✗ НЕМАЄ
On-time Delivery %	—	✗ НЕМАЄ
Завантаження верстатів	—	⚠ ЧАСТКОВО
Ageing деталей у БЗ	—	✗ НЕМАЄ

✓ Найкраще в аналітиці

mes_monthly_report() — сервер-сайдна агрегація. Повертає JSON з: нарядами, кількістю карт, виробленою кількістю, браком, фрезами, матеріалами. Виключає ВБ замовлення. Побудовано через materialized CTE — ефективно.

production_summary_rollup — singleton tabel з тригером. Загальний підрахунок виробленого та браку без full-scan history таблиці. Правильне архітектурне рішення.

✗ Що відсутнє (критично для керівництва)

- **KPI оператора** — немає деталі по продуктивності кожного оператора. Неможливо визначити слабку ланку.
- **On-time delivery** — order.deadline є в БД, але немає функції яка рахує % замовлень виконаних вчасно.
- **Ageing report** — деталі в bz, scrap_ready, wip_bz можуть «застрягти» тижнями. Немає звіту «деталі у БЗ більше X днів».



Розділ 10

Вузькі місця та Технічний борг



Текстовий matching матеріалів (Склад)

Критично

materialInventoryMatching.js — regex парсинг по назвах. Будь-яка орфографічна різниця = помилка матчингу. «Лист Т300 (4мм)» ≠ «лист Т300-4 мм». Зайві замовлення в постачання, зупинки виробництва.



Заморожені wip_bz резервації

Критично

Скасований або видалений наряд не гарантовано звільняє BZ резервацію. Залишок у wip_bz вічно «відсутній» в доступному БЗ. Потрібен щоденний reconciliation job.



789 scratch-файлів у production кодебазі

Серйозно

Директорія /scratch містить 789 файлів-скриптів для ручних виправлень. Це свідчить про часті «аварійні» корекції даних напряму в БД. Кожна така правка — потенційна помилка та незафіксована зміна.



Дублювання модулів (подвійні версії)

Серйозно

MasterModule.jsx + MasterModule_v3.jsx, EngineerModule.jsx + EngineerV2Module.jsx, WarehouseModule + WarehouseModuleV2. Невідомо який активний. Розробнику важко зрозуміти де правити.



Кастомна аутентифікація (не Supabase Auth)

Серйозно

system_users — власна таблиця користувачів. RLS базується на кастомних перевірках, а не на auth.uid(). Ризик: якщо хтось отримає анон ключ — має доступ до даних через RLS «using (true)» policies.



card_info — текстові мета-теги замість FK

Помірно

Зв'язки між картами кодуються як [VKYA_RETURN:uuid:qty] у текстовому полі. Парсинг через regex. Немає constraint'y. Помилка у тексті = втрата зв'язку без error.

**Відсутній Supplier Loop (постачання)****Серйозно**

SupplyModule отримує запити, але немає автоматичного зв'язку між «замовлено у постачальника» і «надійшло на склад». Вся прийомка — ручна операція без РО-підтвердження.



Розділ 11

Рекомендації (за пріоритетом)

● Пріоритет 1 — Критично (виправити негайно)

P1**Перейти на nomenclature_id matching у Складі**

Прибрати текстовий regex matching як основний механізм. Всі запити на матеріали повинні мати обов'язковий nomenclature_id. Це вже підтримується як fallback у inventoryMatchesRequest — зробити його первинним.

P1**Automatic BZ reservation release при скасуванні наряду**

Додати тригер або RPC-виклик при видаленні/скасуванні task → автоматично викликати release_bz_reservation(). Також додати щоденний job який знаходить wip_bz без active task і звільняє їх.

P1**reserved_qty враховувати 'pending' запити**

Тригер reconcile_inventory_reserve має рахувати і pending, і issued статуси. Або додати окремий pending_qty стовпчик в inventory. Без цього два форементи можуть видати один матеріал двічі.

● Пріоритет 2 — Важливо (виправити за тиждень)

P2**Замінити card_info text-теги на proper FK колонки**

Додати колонки vku_source_history_id, redo_source_card_id тощо до work_cards. Міграція даних з regex-парсингу в FK. Це усуне «сліпу зону» відстеження.

P2**Видалити застарілі версії модулів**

Провести audit: який MasterModule є поточним — v3 чи основний? Видалити неактуальний. Аналогічно для EngineerModule, WarehouseModule. Це зменшить confusion і ризик редагування не того файлу.

P2**Додати Ageing Report для БЗ та scrap_ready**

SQL View або RPC: деталі у bz/wip_bz/scrap_ready довше ніж 7/14/30 днів. Сповіщення майстру якщо деталі «зависли».

● Пріоритет 3 — Бажано (виправити за місяць)

P3**KPI оператора та On-time Delivery**

Додати aggregated view по операторах (кількість деталей, брак%, середній час). Рахувати % замовлень виконаних до deadline. Показати в DashboardModule.

P3**Замкнути Supplier Loop**

Додати таблицю purchase_orders з прив'язкою до supply_requests. При прийомці на склад — автоматично close PO і оновлювати inventory. Без ручного введення.

P4**Перевести auth на Supabase Auth**

Довгострокова ціль. Використовувати auth.uid() у RLS policies замість кастомної system_users таблиці. Значно підвищить безпеку.



Розділ 12

Зведена таблиця оцінок по модулях

МОДУЛЬ / ПРОЦЕС	ОЦІНКА	РІВЕНЬ	ГОЛОВНА ПРОБЛЕМА	ГОЛОВНА СИЛЬНА СТОРОНА
База даних (PostgreSQL)	91%	ВІДМІННО	RLS через 'using(true)'	Тригери, ledger, RPC atomicity
БЗ Резервування / Ledger	82%	ДОБРЕ	Осиротілі wip_bz	Атомарний BZ ledger з балансами
Брак / ВКЯ Класифікація	85%	ДОБРЕ	Немає alerting по scrap_ready	4 категорії, DB-level validation
Повернення у наряди	62%	ЗАДОВІЛЬНО	stage_name regexp = risk	Return і Restoration RPC
Склад (Warehouse V2)	71%	ПОМІРНО	Текстовий matching	Типи залишків, pocket_owner
Shop1 Terminal (Розкрій)	68%	ПОМІРНО	Мегакомпонент 278KB	Повна реєстрація браку
Shop2 Terminal (Цех 2)	70%	ПОМІРНО	bz_shop2 без deduction	VKYA Source Route tracking
Аналітика / Звіти	74%	ПОМІРНО	Немає KPI оператора	Monthly report, rollup
Менеджер / Замовлення	70%	ПОМІРНО	CRM новий, не обкатаний	Snapshot system
Постачання (Supply)	55%	ЗАДОВІЛЬНО	Немає PO → прийомка зв'язку	Статуси запитів є
🏆 ЗАГАЛЬНА ОЦІНКА	73%	ФУНКЦІОНАЛЬНА	Matching + wip_bz freeze	DB Architecture + VKYA

✦ Підсумковий висновок

Система **ProjectCentrum** є функціональним MES з добре продуманою архітектурою бази даних. **73% загальна оцінка** означає: система працює і деталі рухаються правильно в основних сценаріях, але є 3 критичних вузькі місця (текстовий matching, заморожені BZ резервації, незамкнений supplier loop), які призводять до помилок обліку та ручних коригувань.

Усунення критичних P1 проблем дозволить підняти оцінку до **88–90%** протягом 2–3 тижнів активної роботи.

 Звіт підготовлено: 29 серпня 2026 р. | ProjectCentrum MES System Audit | Antigravity AI Analysis

Версія аналізу: 1.0 | Базується на 66 міграціях БД, 54 модулях frontend, 789 scratch-скриптах